

Stroiu Alexandra Ioana

Grupa 333AA

Proiect AWJ – Raport

Aplicație pentru gestionarea activităților unui profesor

1. Specificare functionalitati de baza si suplimentare

Aplicația web este realizată cu Spring Boot (back-end) și Thymeleaf (interfață web).

Aplicația permite profesorilor să se autentifice și să își gestioneze activitățile (creare, editare, ștergere, vizualizare).

Datele sunt stocate într-o bază de date relațională (JPA asigură legătura dintre clasele Java și tabelele din baza de date). Aplicația este structurată pe arhitectura MVC și utilizează Spring Security pentru controlul accesului.

Repository: <https://github.com/alexandrastroiu/teacher-activity-manager>

Funcționalități de bază:

- Utilizatorul poate să vizualizeze o colecție de obiecte: cursurile pe care le predă, sesiunile de curs pentru un curs selectat, prezența pentru o sesiune de curs selectată, lista de activități (fiecare activitate poate avea subtask-uri).
- Utilizatorul poate adăuga/șterge/modifica activități din lista de activități
- Utilizatorul poate adăuga/șterge/modifica subtask-uri pentru o activitate din lista de activități
- Utilizatorul poate modifica o sesiune din lista de sesiuni de curs
- Utilizatorul poate adăuga o sesiune în lista de sesiuni de curs
- Pentru o activitate se specifică progresul acesteia în funcție de indeplinirea unor indicatori cheie (progresul unei activități este calculat în funcție de câte subtask-uri ale activității sunt bifate)
- Aplicația afișează mesaje în interfața grafică pentru evitarea ștergerilor accidentale (ștergerea trebuie să fie confirmată de utilizator înainte de a fi realizată)
- Aplicația face verificări de validare a datelor introduse și va afișa mesaje de eroare dacă este cazul

Funcționalități suplimentare:

- Sortare și filtrare a listei de activități (după status, prioritate, dificultate, deadline)
- Grafic (bar chart) cu progresul fiecărei activități
- O secțiune cu progresul curent (numărul total de activități, numărul de activități complete, media progresului)
- Autentificare cu nume de utilizator și parolă pentru profesori (din baza de date) – login și logout

2. Descrierea claselor/metodelor/atributelor proiectului

1. Model

Clasele mapează tabelele din baza de date prin JPA.

- Activity: mapează tabelul activities din baza de date (@Entity @Table(name = "activities"))
- ActivitySubtask: mapează tabelul activity_subtasks din baza de date (@Entity @Table(name = "activity_subtasks"))
- Attendance: mapează tabelul attendance (@Entity @Table(name = "attendance"))
- AttendanceId: clasa pentru cheia compusa (attendance ID)
- Course: mapează tabelul courses (@Entity @Table(name = "courses"))
- CourseEnrollment: mapează tabelul course_enrollment (@Entity @Table(name = "course_enrollment"))
- CourseEnrollmentId: Clasa pentru cheia compusa
- CourseSession: mapează tabelul course_sessions (@Entity @Table(name = "course_sessions"))
- Department: mapează tabelul departments (@Entity @Table(name = "departments"))
- Student: mapează tabelul students (@Entity @Table(name = "students"))
- StudentGroup: mapează tabelul student_groups (@Entity @Table(name = "student_groups"))
- Teacher: mapează tabelul teachers (@Entity @Table(name = "teachers"))
- User: mapează tabelul users (@Entity @Table(name = "users"))

Metode: getters, setters

Atribute: reprezintă câmpurile din tabele (@Column(name = " "))

2. Repository

Clase folosite pentru accesul la baza de date.

- ActivityRepository
Metode:
 - findByTeacher_TeacherId
 - findByTeacher_TeacherIdAndStatus
 - findByTeacher_TeacherIdAndPriority
 - findByTeacher_TeacherIdAndDifficulty
 - findByTeacher_TeacherIdOrderByEndDateAsc
 - countByTeacher_TeacherIdAndStatus
 - countByTeacher_TeacherId
- ActivitySubtaskRepository
Metode:
 - findByActivity_ActivityId
 - countByActivity_ActivityId
 - countByActivity_ActivityIdAndIsCompletedTrue
- AttendanceRepository
Metode:
 - findBySession_SessionId
- CourseEnrollmentRepository
Metode:
 - findByCourseCourseId

- CourseRepository
Metode:
 - findByTeacher_TeacherId
 - countByTeacher_TeacherId
- CourseSessionRepository
Metode:
 - findByCourse_CourseId
 - countByCourseCourseId
- DepartmentRepository
- StudentRepository
- TeacherRepository
Metode:
 - findByUserUserId
- UserRepository
Metode:
 - findByUsername

3. Service

Clase folosite pentru implementarea cazurilor de utilizare.

- ActivityService: Clasa pentru implementarea operatiilor (Create, Update, Delete) si a cazurilor de utilizare (sortare, filtrare, progres) pentru activitati
Attribute: activityRepository, subtaskRepository
Metode: create (creeaza o activitate), delete(șterge o activitate), update(actualizează o activitate), getId(găsește o activitate după ID), findAll, getActivitiesByTeacher(retornează activitățile pentru un profesor), getActivitiesSortedByDeadline(sortează activitățile după deadline), getActivitiesByStatus(sortează activitățile după status), getActivitiesByPriority(sortează activitățile după prioritate), getActivitiesByDifficulty(sortează activitățile după dificultate), calculateProgress(calculează progresul pentru o activitate), countCompletedActivities(retornează numărul de activități complete), countTotalActivities(retornează numărul total de activități), getAverageProgress (calculează media progresului pentru toate activitățile)
- ActivitySubtaskService: Clasa pentru implementarea operatiilor (Create, Update, Delete) si a cazurilor de utilizare pentru subtask-uri
Attribute: subtaskRepository, activityRepository
Metode: create(creează un subtask), findAll, setCompleted (completează un subtask), delete(șterge un subtask), getSubtasksByActivity(retornează subtask-urile unei activități), updateTitle(actualizează titlul unui subtask)
- AttendanceService: Clasa pentru implementarea operatiilor (Create, Update, Delete) si a cazurilor de utilizare pentru prezenta
Attribute: attendanceRepository, sessionRepository, studentRepository
Metode: markAttendance (actualizează sau creează prezența), deleteAttendance (șterge prezența pentru o sesiune de curs), getAttendanceForSession (retornează prezența pentru o sesiune de curs), attendanceExists (verifică dacă o înregistrare pentru prezență există deja)
- CourseService: Clasa pentru implementarea cazurilor de utilizare pentru cursuri
Attribute: courseRepository, sessionRepository

Metode: getById (găsește cursul după ID), getCoursesByTeacher (returnează cursurile pe care le predă un profesor), countSessionsForCourse (returnează numărul de sesiuni pentru un curs)

- CourseSessionService: Clasa pentru implementarea operatiilor (Create, Update, Delete) pentru sesiuni de curs
Attribute: sessionRepository
Metode: getSessionsByCourse(returnează sesiunile unui curs), createSession, updateSession, deleteSession, getById (găsește sesiunea după ID)
- StudentService: Clasa pentru implementarea cazurilor de utilizare pentru student
Attribute: studentRepository, enrollmentRepository
Metode: getStudentsByCourse (găsește studenții înscriși la un curs)
- TeacherService : Clasa pentru implementarea cazurilor de utilizare pentru profesori
Attribute: teacherRepository, courseRepository
Metode: getByUserId (găsește profesorul după ID-ul de utilizator), getTeacherWithDepartment, countCoursesForTeacher (returnează numărul de cursuri pentru un profesor)
- UserService: Clasa pentru logica de autentificare a unui utilizator
Attribute: userRepository, passwordEncoder
Metode: findByUsername (găsește utilizatorul după numele de utilizator), authenticateTeacher (metodă pentru logica de login)

4. Controller

Clase care apelează clasele service, preiau cereri HTTP și returnează răspunsuri (pagini Thymeleaf sau JSON).

- ActivityController (@RestController)
- ActivitySubtaskController (@RestController)
- AttendanceController (@RestController)
- AuthController (@Controller): clasa controller pentru autentificare
- CourseSessionController (@RestController)
- DashboardWebController: Clasa controller pentru interfata grafica (@Controller)
- TeacherController (@RestController)
- UserController (@RestController)

5. DTO (Data Transfer Objects)

Clase folosite pentru transferul datelor între controller și client.

- ActivityCreatedto: clasă pentru transferul datelor de la client catre API si validarea datelor pentru a crea o activitate
Attribute și validări: title(@NotBlank, @Size(min = 3, max = 80)), description(@Size(max = 300)), startDate, endDate(@FutureOrPresent), status(@NotNull), priority(@NotNull), difficulty(@NotNull)
- ActivityResponseDto: clasă pentru transferul datelor de raspuns catre client pentru activitati
Attribute: activityId, title, description, startDate, endDate, status, priority, difficulty, progress, subtasks
- ActivityUpdatedto: clasă pentru transferul datelor de la client catre API si validarea datelor pentru a actualiza o activitate
Attribute: title(@NotBlank, @Size(min = 3, max = 80)), description(@Size(max = 300)), startDate, endDate(@Size(max = 300)), status(@NotNull), priority(@NotNull), difficulty(@NotNull)

- AttendanceCreateDto: clasă pentru transferul datelor de la client catre API si validarea datelor pentru a crea prezența
Attribute: sessionId(@NotNull), studentId(@NotNull), status(@NotNull)
 - AttendanceResponseDto: clasă pentru transferul datelor de raspuns catre client pentru prezența
Attribute: sessionId, studentId, , studentName, groupName, status
 - AttendanceUpdateDto: clasă pentru transferul datelor de la client catre API si validarea datelor pentru a actualiza prezența
Attribute: status(@NotNull)
 - CourseResponseDto: clasă pentru transferul datelor de raspuns catre client pentru cursuri
Attribute: courseId, courseName
 - CourseSessionResponseDto: clasă pentru transferul datelor de raspuns catre client pentru sesiuni de curs
Attribute: sessionId, courseId, courseName, sessionDate, sessionTime, duration
 - CreateCourseSessionDto: clasă pentru transferul datelor de la client la API si validarea datelor pentru a crea o sesiune de curs
Attribute: courseId(@NotNull, @Positive), sessionDate(@NotNull, @FutureOrPresent), sessionTime(@NotNull), duration(@NotNull)
 - LoginRequestDto: clasă pentru transferul datelor de autentificare de la client catre API
Attribute: username(@NotBlank), password(@NotBlank)
 - StudentResponseDto: clasă pentru transferul datelor de raspuns catre client pentru student
Attribute: studentId, firstName, lastName, groupName
 - SubtaskCreateDto: clasă pentru transferul datelor de la client catre API si validarea datelor pentru a crea un subtask
Attribute: title(@NotBlank, @Size(min = 2, max = 50))
 - SubtaskResponseDto: clasă pentru transferul datelor de raspuns catre client pentru subtask-uri
Attribute: subtaskId, title, completed
 - TeacherDashboardDto: clasă pentru transferul datelor de raspuns catre client pentru dashboard
Attribute: totalActivities, completedActivities, averageProgress
 - UpdateCourseSessionDto: clasă pentru transferul si validarea datelor pentru actualizarea unei sesiuni de curs de la client catre API
Attribute: sessionDate(@NotNull), sessionTime(@NotNull), duration(@NotNull)
 - UserResponseDto: clasă pentru transferul datelor de raspuns catre client pentru utilizatori
Attribute: userId, username, email, role
- Metode: getters, setters, from (metodă care convertește o entitate într-un obiect de răspuns)

6. Clase pentru logica de autentificare

- SecurityConfig: clasă pentru configurarea autentificării (login, logout, criptarea parolelor)
Metode: passwordEncoder (metodă folosită pentru verificarea parolei la login), securityFilterChain (definește permisiunile de acces la rurile aplicației)
- TeacherUserDetailsService

Metode: loadUserByUsername (caută utilizatorul în baza de date și restricționează accesul pentru utilizatorii care nu au rolul de profesor)

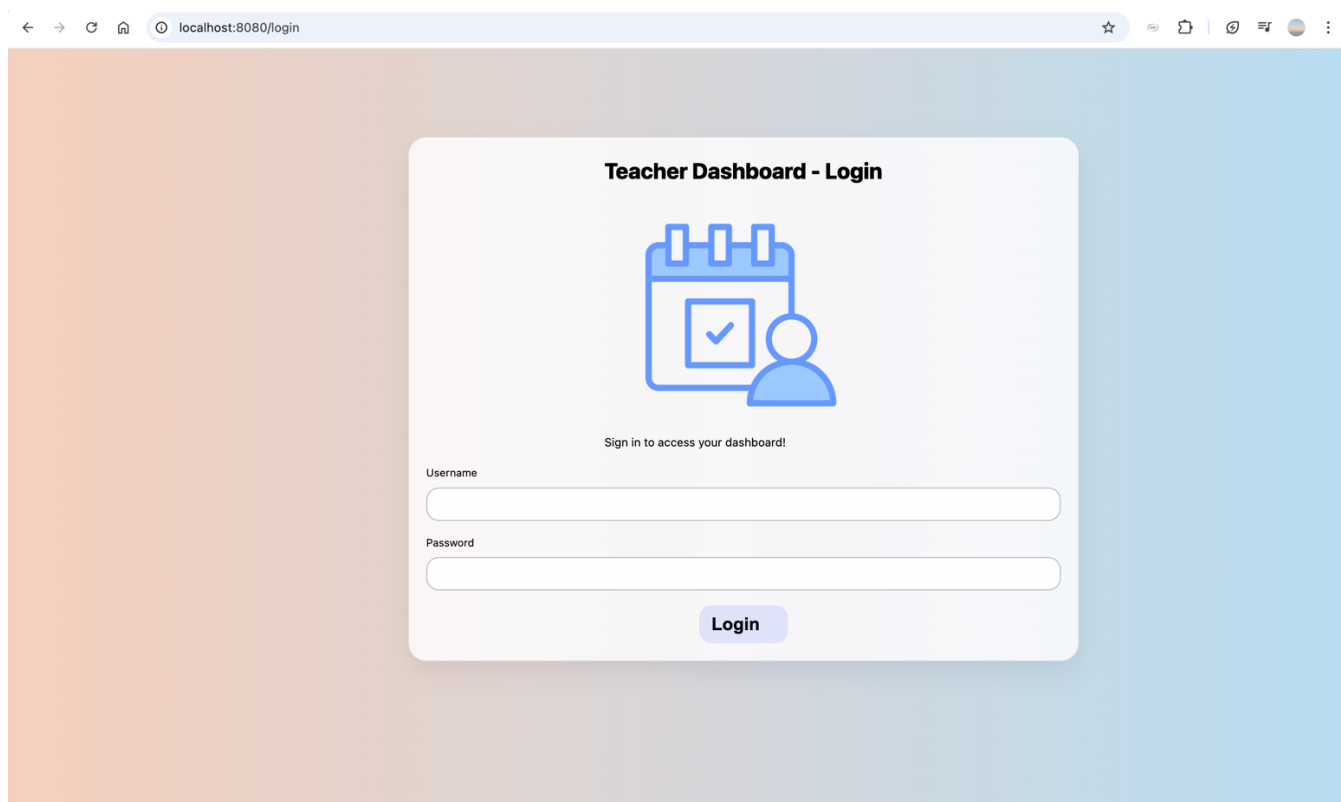
Atribute: userRepository (atribut folosit pentru accesul la baza de date, pentru a găsi un utilizator după numele de utilizator)

3. Descrierea elementelor functionale oferite prin interfata grafica cu utilizatorul

Interfața grafică conține două pagini: pagina de autentificare (login) și dashboard (pagina principală) realizate folosind Thymeleaf și CSS pentru stilul paginilor.

Pagina de autentificare (Login):

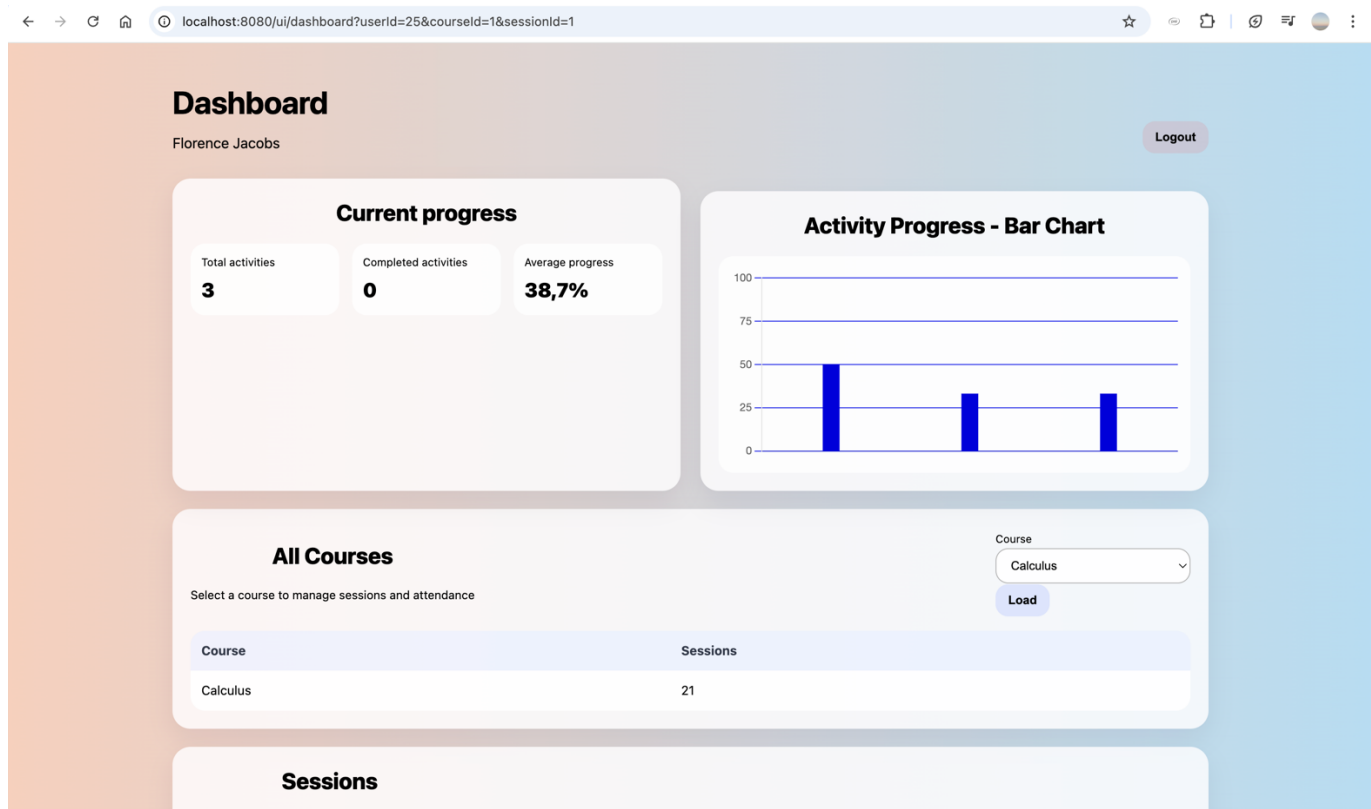
- Formular de login: câmpuri pentru nume de utilizator și parolă
- Buton de login
- Mesaje de eroare (Nume de utilizator/parolă invalidă sau rolul nu este profesor)



Dashboard (pagina principală)

- Afișează numele profesorului logat
- Progresul curent (numărul total de activități, numărul de activități complete, media progresului)
- Grafic cu progresul fiecărei activități (realizat cu Javascript)
- Lista cursurilor pe care le predă și numărul de sesiuni de curs
- Lista cu sesiuni de curs (cursul se selctează din dropdown)

- Buton pentru editarea unei sesiuni de curs (se apasă butonul Edit, se aleg valori noi pentru câmpurile Date, Time, Duration, se apasă butonul Save Session sau butonul Cancel)
- Buton pentru selectarea unei sesiuni de curs (Select)
- Buton pentru a adăuga o noua sesiune (Add session) și câmpuri (Date, Time, Duration)
- Lista prezenței pentru o sesiune de curs selectată (Prezența poate fi modificată – se selectează Present sau Absent din dropdown si se apasă butonul Save)
- Lista de activități (All activities)
- Se poate crea o activitate (se completează toate câmpurile Title, Description, Start, End, Status, Priority, Difficulty și se apasă butonul Create)
- Se poate edita o activitate (se selectează din dropdown, se apasă butonul Load, se completează toate câmpurile, se apasă butonul Save sau butonul Cancel pentru anulare)
- Butonul delete pentru a șterge o activitate
- Se completează câmpul „New subtask” cu titlul noului subtask și se apasă butonul Add
- Se poate modifica titlul unui subtask din interfața grafică (se salvează cu butonul Save)
- Se poate marca un subtask (click pe checkbox)
- Se șterge un subtask apăsând butonul X



← → ↺ 🏠 🔍

localhost:8080/ui/dashboard?userId=25&courseId=1&sessionId=1

☆ 🔄 📄 🗑️ 📝 ⌵

Sessions

Add / Edit / Delete sessions for the selected course

Date

dd.mm.yyyy 📅

Time

--:-- ⌚

Duration

--:-- ⌚

Add session

Date	Time	Duration	Actions	
2025-09-02	10:00	02:00	Select	Edit
2025-09-09	10:00	02:00	Select	Edit
2025-09-16	10:00	02:00	Select	Edit
2025-09-23	10:00	02:00	Select	Edit
2025-09-30	10:00	02:00	Select	Edit
2025-10-07	10:00	02:00	Select	Edit
2025-10-14	10:00	02:00	Select	Edit
2025-10-21	10:00	02:00	Select	Edit
2025-10-28	10:00	02:00	Select	Edit
2025-11-04	10:00	02:00	Select	Edit

← → ↺ 🏠 🔍

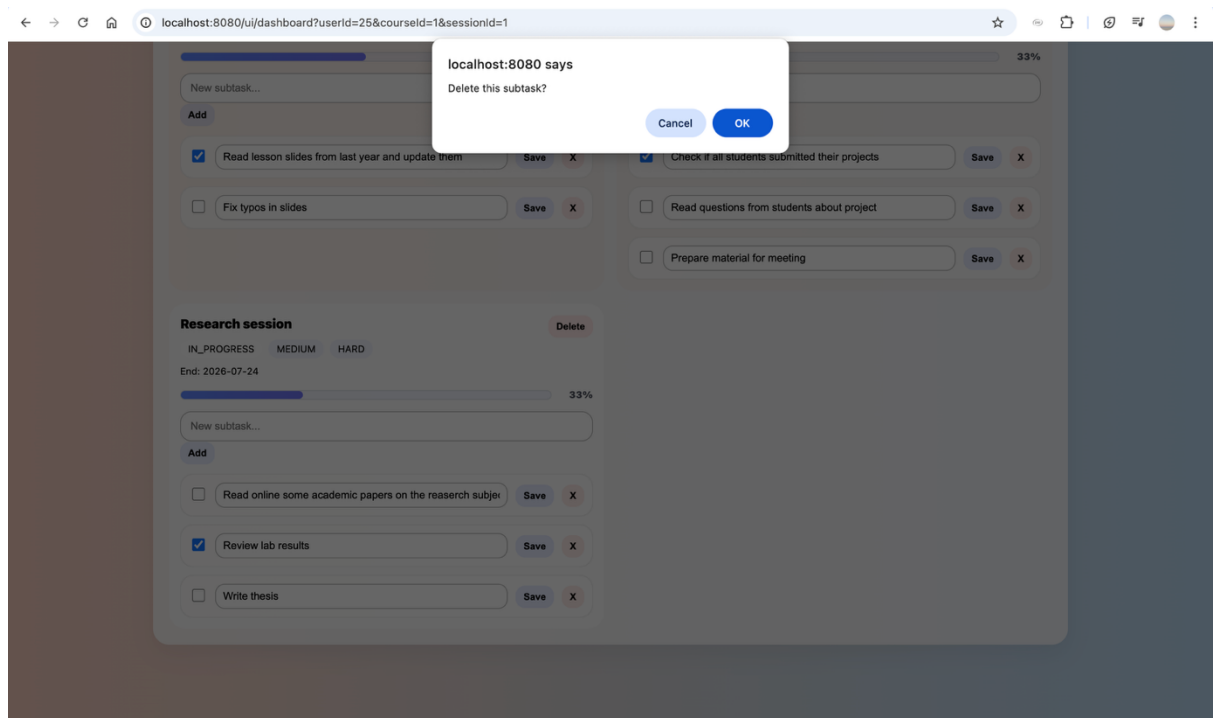
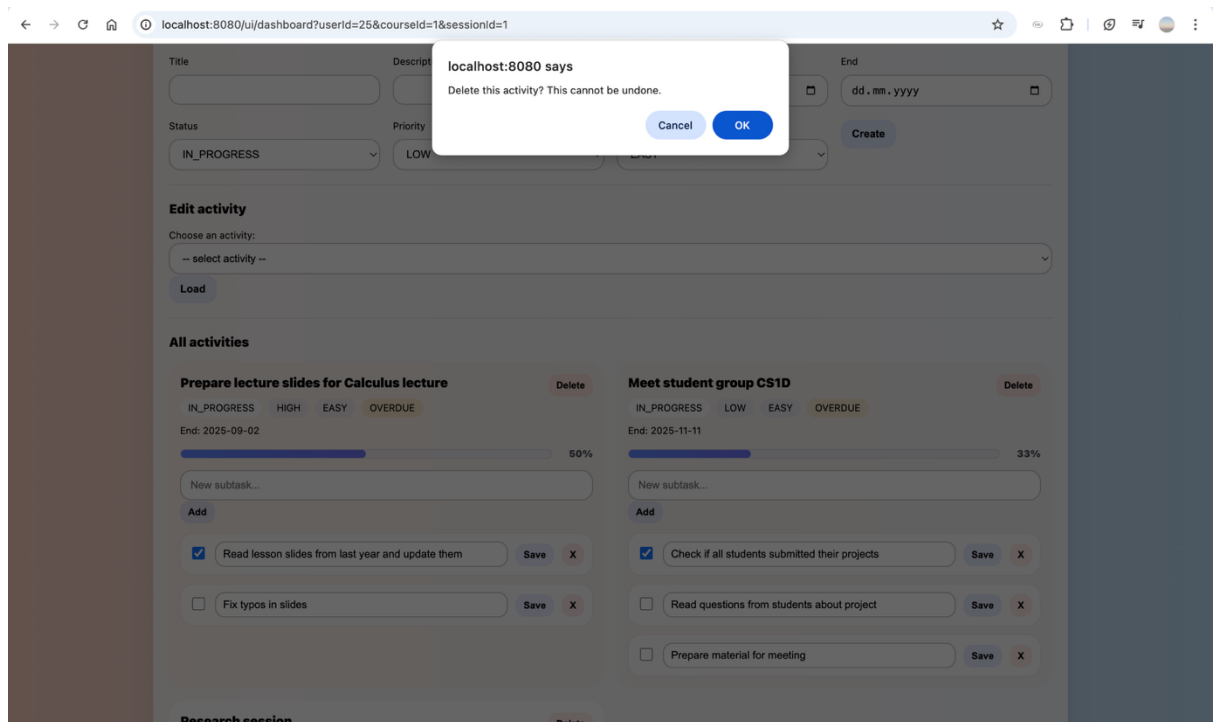
localhost:8080/ui/dashboard?userId=25&courseId=1&sessionId=1

☆ 🔄 📄 🗑️ 📝 ⌵

Mark attendance for selected session

All students enrolled in the selected course:

Student	Group	Status
Rachel Holland	CS1A	ABSENT Save
Alice Brown	CS1A	PRESENT Save
Carol Smith	CS1B	PRESENT Save
Melanie Rogers	CS1B	PRESENT Save
Lydia Robinson	CS1B	PRESENT Save
Evan Anderson	CS1C	PRESENT Save
Braxton Johnson	CS1C	PRESENT Save



Activities

Create/ Filter/ Edit/ Delete activities and subtasks.

Status

Priority

Difficulty

Sort

All

All

All

None

Apply

Create activity

Title

Description

Start

End

New activity

dd . mm . yyyy

dd . mm . yyyy

Status

Priority

Difficulty

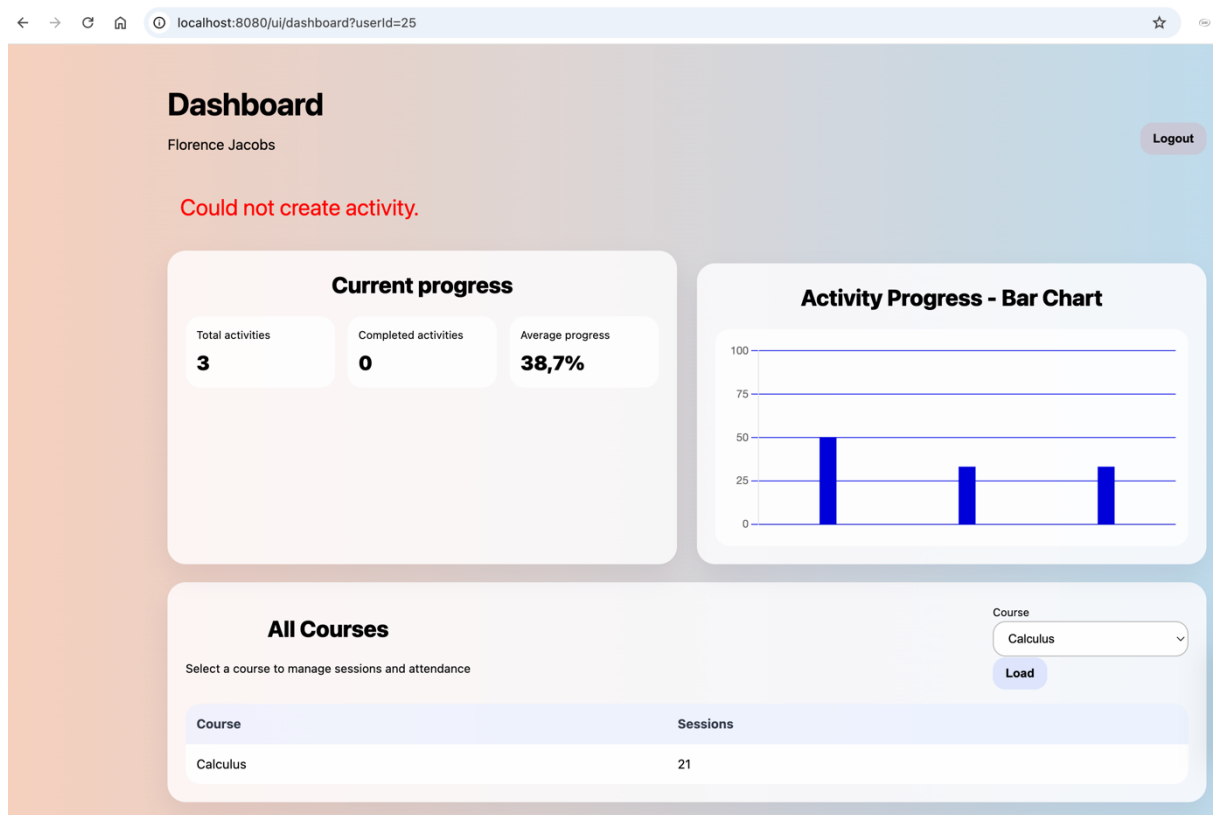
Create

IN_PROGRESS

MEDIUM

MEDIUM

Mesaj de eroare (câmpurile sunt incomplete, nu poate fi creată o nouă activitate):



Mesaj de eroare (nu se poate marca completă o activitate care nu are progres 100%):

Edit activity

Choose an activity:

Prepare lecture slides for Calculus lecture

Load

Title

Description

Start

End

Prepare lecture slides for Calculus lectu

Read lecture slides and prepare lesson

dd . mm . yyyy

dd . mm . yyyy

Status

Priority

Difficulty

Save

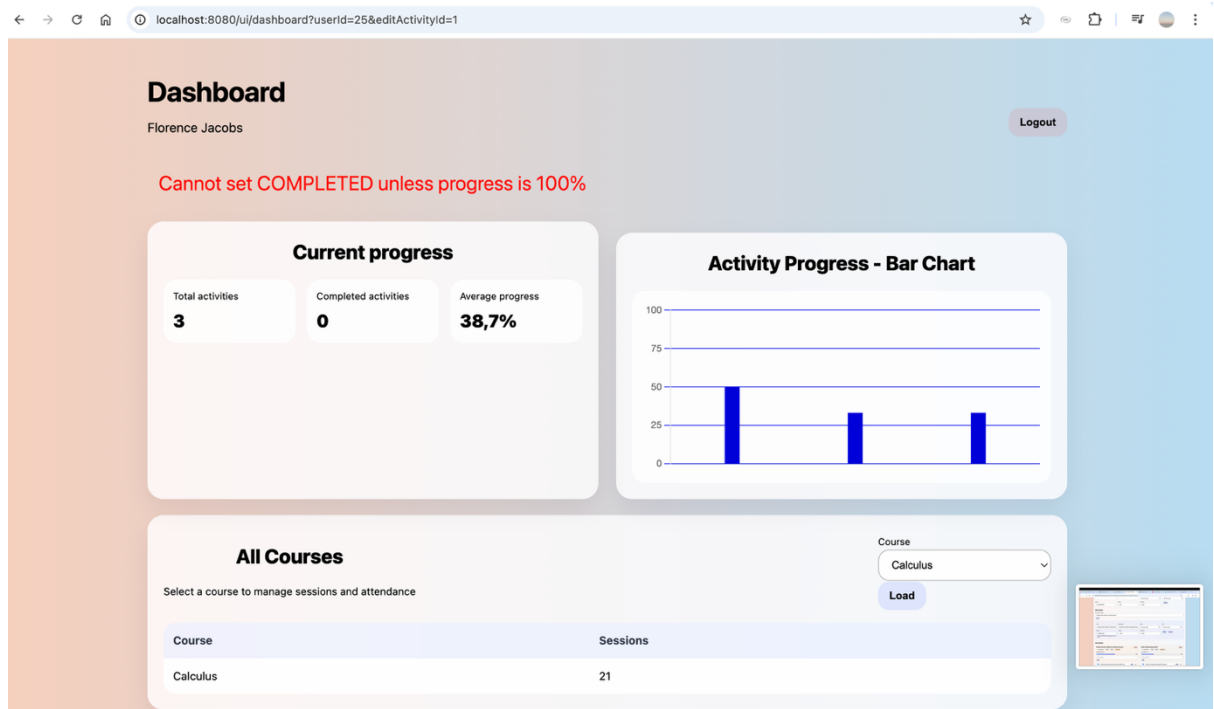
Cancel

COMPLETED

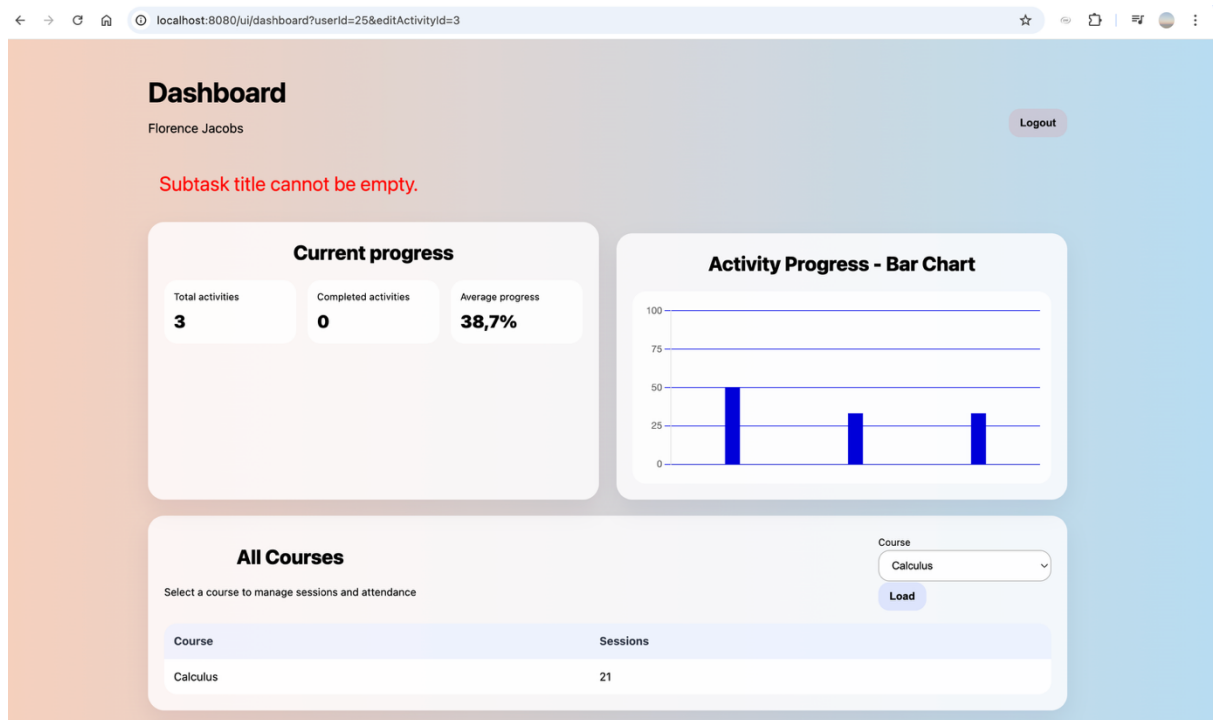
HIGH

EASY

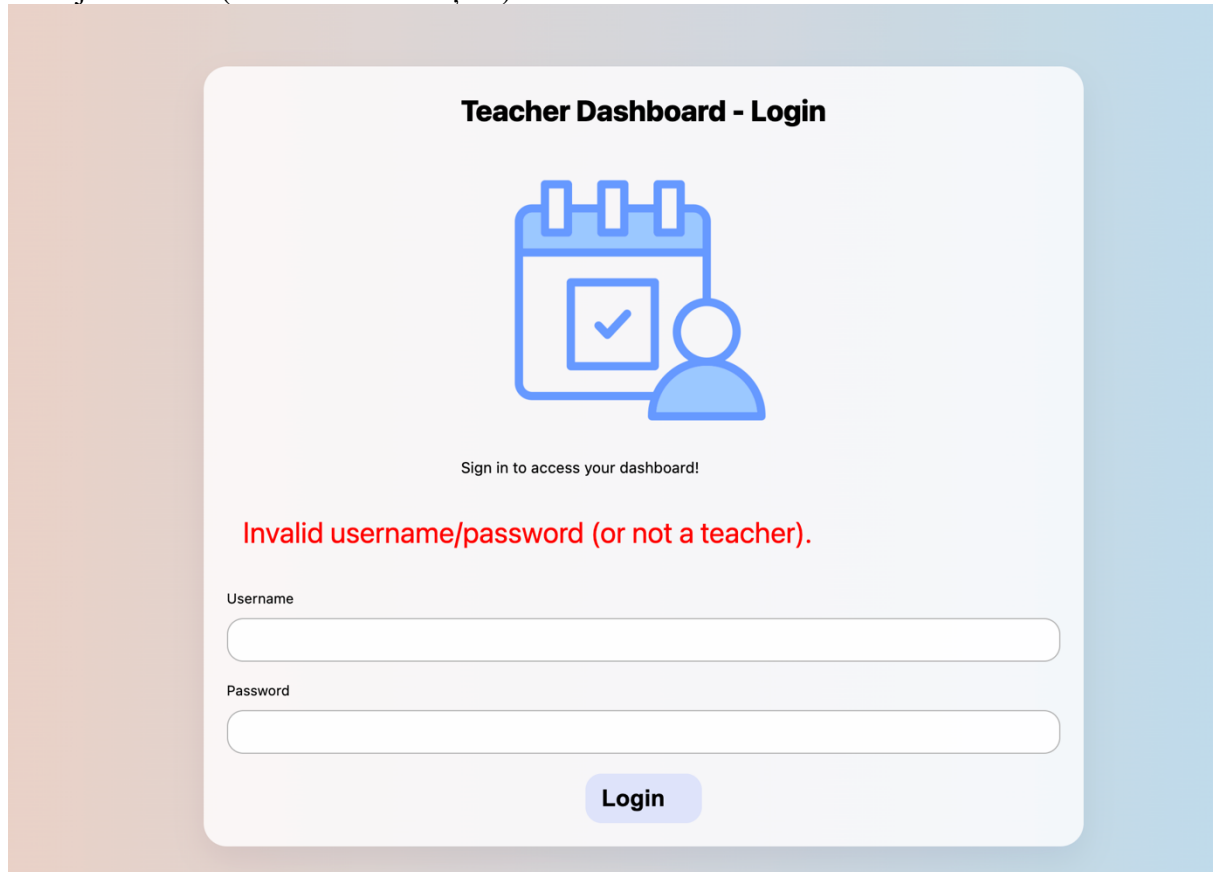
If you set COMPLETED, progress must be 100%.



Mesaj de eroare (titlu e incomplet, nu poate fi creat un nou subtask):



Mesaj de eroare (autentificarea a eșuat):

The image shows a login form titled "Teacher Dashboard - Login". At the top, there is a blue icon of a calendar with a checkmark and a person silhouette. Below the icon, the text "Sign in to access your dashboard!" is displayed. A red error message, "Invalid username/password (or not a teacher).", is shown above the input fields. There are two input fields: "Username" and "Password". At the bottom, there is a blue "Login" button.

Teacher Dashboard - Login

Sign in to access your dashboard!

Invalid username/password (or not a teacher).

Username

Password

Login

4. Aspecte privind testarea aplicatiei

Am testat aplicația la nivel de API și la nivel de interfață web.

Am verificat endpoint-urile REST cu Postman:

- Am verificat operațiunile de adăugare, ștergere, editare
- Am verificat codul de răspuns HTTP și conținutul JSON returnat
- Am verificat cazuri de eroare: ID inexistent, câmpuri obligatorii lipsă

Am testat manual interfața web în browser:

- Autentificare cu date valide și invalide
- Testarea logout-ului
- Operații de ștergere, editare, adăugare pentru activități și subtask-uri
- Editare și adăugare pentru sesiuni de curs
- Modificarea prezenței
- Verificarea validărilor din formulare (câmpuri obligatorii, date incorecte)
- Am testat că progresul se modifică și este afișată valoarea corectă

Postman interface showing a GET request to `/api/activities/teacher/{{teacherid}}`. The response is a 200 OK status with a JSON body containing an array of activities. The first activity is:

```
1 {
2   {
3     "activityId": 1,
4     "title": "Prepare lecture slides for Calculus lecture",
5     "description": "Read lecture slides and prepare lesson plan for Chapter 1 lecture on Monday",
6     "startDate": "2025-08-30",
7     "endDate": "2025-09-02",
8     "status": "IN_PROGRESS",
9     "priority": "HIGH",
10    "difficulty": "EASY",
11    "progress": 50,
12    "subtasks": [
13      {
14        "subtaskId": 1,
15        "title": "Read lesson slides from last year and update them",
16        "completed": true
17      },
18      {
19        "subtaskId": 26,
20        "title": "Fix typos in slides",
21        "completed": false
22      }
23    ]
24  },
25  {
26    "activityId": 2,
27    "title": "Meet student group CS1D",
28    "description": "Meet student group CS1D for project presentation session",
29    "startDate": null,
30    "endDate": "2025-11-11",
```

Postman interface showing a GET request to `/api/teachers/user/{{userid}}/dashboard`. The response is a 200 OK status with a JSON body containing summary statistics:

```
1 {
2   "totalActivities": 3,
3   "completedActivities": 0,
4   "averageProgress": 38.666666666666664
5 }
```

The bottom status bar shows: Cloud View, Find and replace, Console, Terminal, Runner, Start Proxy, Cookies, Vault, Trash, and a help icon.

Search Postman

DEL Delet [CONFLIK GET Filter GET Filter GET {{bas DEL Delet DEL Delet DEL Delet PUT Edit : GET List si

Test / List subtasks

GET {{baseUrl}}/api/activities/{{activityId}}/subtasks

Send

Docs Params Authorization Headers

3

Collection Variables in request

Key	Value	Description	Bulk Edit	Presets
☒ Content-Type	application/json			
Key	Value	Description		

Body Cookies (1) Headers (11) Test Results

200 OK • 11 ms • 557 B

Save Response

JSON Preview Visualize

```
1 [
2   {
3     "subtaskId": 3,
4     "title": "Read online some academic papers on the reaserch subject",
5     "completed": false
6   },
7   {
8     "subtaskId": 14,
9     "title": "Review lab results",
10    "completed": true
11  },
12  {
13    "subtaskId": 27,
14    "title": "Write thesis",
15    "completed": false
16  }
17 ]
```

Runner Start Proxy Cookies Vault Trash

Search Postman

POST Cre. [CONFLIK GET Verify POST Cre. GET List si

Test / List sessions for course

GET {{baseUrl}}/api/sessions/course/{{courseId}}

Send

Docs Params Auth Headers (8) Body Scripts Settings

Headers

Key	Value	D...	Bulk Edit	Presets
☒ Content-Type	application/json			
Key	Value	Description		

Body

200 OK • 32 ms • 2.94 KB

Save Response

JSON Preview Visualize

```
1 [
2   {
3     "sessionId": 1,
4     "courseId": 1,
5     "courseName": "Calculus",
6     "sessionDate": "2025-09-02",
7     "sessionTime": "10:00:00",
8     "duration": "02:00:00"
9   },
10  {
11    "sessionId": 2,
12    "courseId": 1,
13    "courseName": "Calculus",
14    "sessionDate": "2025-09-09",
15    "sessionTime": "10:00:00",
16    "duration": "02:00:00"
17  },
18  {
19    "sessionId": 3,
20    "courseId": 1,
21    "courseName": "Calculus"
22  }
23 ]
```

New Chat

Create a new collection for activities API

Generate documentation for activities endpoint

Build test suite for activities API

GET List sessions for ...

Describe what you need. Press @ for context, / for Skills.

Auto

Runner Start Proxy Cookies Vault Trash

5. 5 idei despre cum poate fi imbunatatita aplicatia

- Adăugarea unei pagini de înregistrare, cu validări pentru parolă, email și nume de utilizator
- Mai multe validări și mesaje mai clare în interfața grafică
- Căutarea unei activități după titlu
- Export CSV/PDF pentru statisticile din dashboard
- Notificări în aplicație pentru activități care urmează să depășească termenul limită